

Documentação: Roteiro 8

5.1 Título

Trabalho Prático: Analisador Léxico - Roteiro 8 (Loops de Repetição: WHILE, FOR, DO-WHILE)

5.2 Visão Geral

Tendo resolvido a questão de controle de escopo do IF e ELSE, o projeto saltou para a criação de laços iterativos ("Loops"). Como a natureza do nosso analisador é unidirecional top-down, a simulação sintática avaliou as repetições sem repetir a leitura de caracteres de fato. Foram adicionados e destrinchados todos os 3 grandes blocos primitivos do JAVA: `while(condicao){}, do{ }while(condicao)` e `for(inicio; limite; passo){}`.

5.3 Tokens e Expressões Regulares

5.3.1 Tabela de Tokens Válidos

Nenhum token visualmente novo a não ser as palavras-chaves descritas abaixo.

5.3.2 Palavras Reservadas

Palavra Reservada	Token Gerado
<code>while</code>	<code>KW_WHILE</code>
<code>for</code>	<code>KW_FOR</code>
<code>do</code>	<code>KW_DO</code>

5.3.3 Autômato Finito Determinista (AFD)

Inseriu-se as 3 chaves sequenciais para compor os saltos AFD, asilando sua interpretação perante à regra `IDENT`.

5.4 Resultados

O analisador aceitou elegantemente construções densas como `for(i=0; i<10; i = i+1)`. Durante o teste da regra `for_stmt`, o sistema desmembra as condições em 3 ramos não terminais (`for_init`, `condicao`, `for_step`), todas atreladas dentro da assinatura mestre exigindo parênteses e dois pontos e vírgulas internos (`PTVIRG`). O bloco interior só tem sua visibilidade aceita na tela caso o bloco limitador `condicao` repasse seu valor lógico via boolean, usando os poderes implementados no roteiro 7.

5.5 Conclusão

O JCup demonstrou excelente separação de contexto entre estruturas parecidas que partilham dos mesmos literais. A regra do `do-while` destacou-se ao provar que o compilador, de fato, precisava forçar e assumir uma execução primária (puxando o `deveExecutarBloco` para `true`) independentemente de conhecer sua trava relacional.

5.6 Referências Bibliográficas

- HAUCK, Alessandra. *Roteiro de Aula Prática - Compiladores (Roteiro 8)*.